Surrogate keys are generated independently of the current row data, so table column constraints may evolve with time (changing a user birthday or email address). The surrogate key can be generated by a numerical sequence generator (e.g. a database identity column or a sequence), or it can be constructed by a pseudorandom number generator (e.g. GUID[3] or UUID[4]). Both the numerical and UUID keys have both pros and cons.

The UUID is defined by the RFC 4122[5] standard and it is stored as a 128-bit sequence. The GUID term refers to any globally unique identifier, which might comprise other non-standard implementations. For consistency, this chapter further refers to unique identifiers as UUID.

A UUID takes 128 bits, which is four times more than an `INTEGER` and twice as `BIGINT`. On the other hand, a UUID number has less chance of a conflict in a Multi-Master database replication topology. To avoid such conflicts, many relational database systems increment the identity or sequence numbers in steps, each node getting its own offset. Because UUIDs are not sequential, they induce fragmentation, and that can really affect the performance of clustered indexes.

Requiring less space and being more index-friendly, numerical sequences are preferred over UUID keys.

9.2.1 UUID identifiers

Nevertheless, some enterprise systems use UUID primary keys, so it is worth knowing what Hibernate types work best for this task. The UUID key can either be generated by the application using the `java.util.UUID` class or it can be assigned by the database system.

If the database system does not have a built-in UUID type, a `BINARY(16)` column type is preferred. Although a `CHAR(32)` column could also store the UUID textual representation, the additional space overhead makes it a less favorable pick.

[3] http://en.wikipedia.org/wiki/Globally_Unique_Identifier
[4] http://en.wikipedia.org/wiki/Universally_Unique_Identifier
[5] https://www.ietf.org/rfc/rfc4122.txt

Oracle

There is no UUID type in Oracle, so a `RAW(16)` column must be used instead. The `SYS_GUID()`[a] database function can generate a globally unique identifier.

[a] http://docs.oracle.com/database/121/SQLRF/functions202.htm#SQLRF06120

SQL Server

The `uniqueidentifier`[a] data type is used for storing GUID identifiers. The `NEWID()`[b] function can generate a UUID compatible with the RFC 4122 standard.

Because by default SQL Server uses clustered indexes for primary keys, to avoid the fragmentation effect, the `NEWSEQUENTIALID()`[c] function can assign pseudo-sequential UUID numbers (greater than previously generated ones). This guarantee is kept as long as the Windows server is not restarted.

[a] https://msdn.microsoft.com/en-us/library/ms187942.aspx
[b] https://msdn.microsoft.com/en-us/library/ms190348.aspx
[c] https://msdn.microsoft.com/en-us/library/ms189786.aspx

PostgreSQL

The `UUID` type[a] can store RFC 4122 compliant unique identifiers. The database does not offer a built-in UUID generation function, so the identifier must be generated by the data access layer.

[a] http://www.postgresql.org/docs/current5/static/datatype-uuid.html

MySQL

The UUID must be stored in a `BINARY(16)` column type. The `UUID()`[a] functions can generate a 128-bit unique identifier. Because the `UUID()` function might cause problems for statement-based replication, passing the generated identifier as a variable[b] is a workaround to this limitation.

[a] http://dev.mysql.com/doc/refman/5.7/en/miscellaneous-functions.html#function_uuid
[b] https://dev.mysql.com/doc/refman/5.7/en/replication-features-functions.html

For generating UUID identifiers, Hibernate offers three generators (*assigned*, *uuid*, *uuid2*), which we'll be discussed in greater detail in the following sections.

9.2.1.1 The assigned generator

By simply omitting the `@GeneratedValue`[6] annotation, Hibernate falls back to the assigned identifier, which allows the data access layer to control the identifier generation process. The following example maps a `java.util.UUID` identifier to a `BINARY(16)` column type:

```
@Entity @Table(name = "post")
public class Post {

    @Id @Column(columnDefinition = "BINARY(16)")
    private UUID id;

    public Post() {}

    public Post(UUID id) {
        this.id = id;
    }
}
```

When persisting a *post*, Hibernate generates the following insert statement:

```
INSERT INTO post (id) VALUES
([86, 10, -104, 26, 60, -115, 79, 78, -118, -45, 64, 94, -64, -40, 66, 100])
```

The `UUIDBinaryType` translates the `java.util.UUID` to an array of `byte(s)` that is stored in the associated `BINARY(16)` column type.

Because the identifier is generated in the data access layer, the database server is freed from this responsibility, and so it can allocate its resources to other data processing tasks.

Hibernate can also generate a UUID identifier on behalf of the application developer, as described in the following two sections.

[6] https://docs.oracle.com/javaee/7/api/javax/persistence/GeneratedValue.html

9.2.2 The legacy UUID generator

The UUID hex generator[7] is registered under the `uuid` name and generates hexadecimal UUID string representations. Using a 8-8-4-8-4 hex digit layout, the UUID hex generator is not compliant with the RFC 4122 standard, which uses a 8-4-4-4-12 hex digit format. The following code snippet depicts the `UUIDHexGenerator` mapping and the associated insert statement.

```
@Entity @Table(name = "post")
public class Post {

    @Id @Column(columnDefinition = "CHAR(32)")
    @GeneratedValue(generator = "uuid")
    @GenericGenerator(name = "uuid", strategy = "uuid")
    private String id;
}
```

```
INSERT INTO post (id) VALUES (402880e451724a820151724a83d00000)
```

9.2.2.1 The newer UUID generator

The newer UUID generator[8] is RFC 4122 compliant (variant 2) and is registered under the `uuid2` name (working with `java.lang.UUID`, `byte[]` and `String` Domain Model object types). Compared to the previous use case, the mapping and the test case look as follows:

```
@Entity @Table(name = "post")
public class Post {

    @Id @Column(columnDefinition = "BINARY(16)")
    @GeneratedValue(generator = "uuid2")
    @GenericGenerator(name = "uuid2", strategy = "uuid2")
    private UUID id;
}
```

```
INSERT INTO post (id) VALUES
([77, 2, 31, 83, -45, -98, 70, 40, -65, 40, -50, 30, -47, 16, 30, 124])
```

[7] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/id/UUIDHexGenerator.html
[8] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/id/UUIDGenerator.html

Being RFC 4122 compliant and able to operate with `BINARY` column type, the `UUIDGenerator` is preferred over the legacy `UUIDHexGenerator`.

9.2.3 Numerical identifiers

As previously explained, a numerical surrogate key is usually preferred since it takes less space and indexes work better with sequential identifiers. To generate numerical identifiers, most database systems offer either identity (or auto_increment) columns or sequence objects.

JPA defines the `GenerationType`[9] enumeration for all supported identifier generator types:

- `IDENTITY` is for mapping the entity identifier to a database identity column.
- `SEQUENCE` allocates identifiers by calling a given database sequence.
- `TABLE` is for relational databases that do not support sequences (e.g. MySQL 5.7), the *table* generator emulating a database sequence by using a separate table.
- `AUTO` decides the identifier generation strategy based on the current database dialect.

As explained in the JDBC part, database sequences work better with batch updates and allow various application-side optimization techniques as well.

9.2.3.1 Identity generator

The identity column type (included in the SQL:2003[10] standard) is supported by Oracle 12c[11], SQL Server[12] and MySQL (AUTO_INCREMENT)[13], and it allows an `INTEGER` or a `BIGINT` column to be auto-incremented on demand.

The incrementation process is very efficient since it uses a lightweight locking mechanism, as opposed to the more heavyweight transactional course-grain locks. The only drawback is that the newly assigned value can only be known after executing the actual insert statement.

[9] https://docs.oracle.com/javaee/7/api/javax/persistence/GenerationType.html
[10] http://en.wikipedia.org/wiki/SQL:2003
[11] http://docs.oracle.com/database/121/SQLRF/statements_7002.htm#SQLRF55657
[12] http://msdn.microsoft.com/en-us/library/ms186775.aspx
[13] http://dev.mysql.com/doc/refman/5.7/en/example-auto-increment.html

Batch updates

Because Hibernate separates the id generation from the actual entity insert statement, entities using the identity generator may not participate in JDBC batch updates. Hibernate issues the insert statement during the `persist()` method call, therefore breaking the transactional write-behind caching semantic used for entity state transitions.

Even if some JDBC drivers allow fetching the associated generated keys when executing a batch update, Hibernate still needs an improvement in this regard.

The identity generator can be mapped as follows:

```
@Entity @Table(name = "post")
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
}
```

The following example demonstrates how the transaction write-behind caching model is circumvented by the identity column semantics. Although disabled by default, JDBC batching was enabled to compare results between identity and sequence generators.

```
doInJPA(entityManager -> {
    for (int i = 0; i < batchSize; i++) {
        entityManager.persist(new Post());
    }
    LOGGER.debug("Flush is triggered at commit-time");
});
```

Executing the previous test case generates the following output.

```
INSERT INTO post (id) VALUES (DEFAULT)
INSERT INTO post (id) VALUES (DEFAULT)

DEBUG - Flush is triggered at commit-time
```

Because the associated entity identifier can only be known after the insert statement is executed, Hibernate triggers the entity state transition prior to flushing the currently running Persistence Context.

9.2.3.2 Sequence generator

A sequence is a database object that generates consecutive numbers. Defined by the SQL:2003 standard, database sequences are supported by Oracle, SQL Server 2012 and PostgreSQL, and, compared to identity columns, sequences offer the following advantages:

- The same sequence can be used to populate multiple columns, even across tables.
- Values may be preallocated to improve performance.
- Allowing incremental steps, sequences can benefit from application-level optimization techniques.
- Because the sequence call can be decoupled from the actual insert statement, Hibernate does not disable JDBC batch updates.

To demonstrate the difference between the identity and the sequence identifier generators, the previous example is changed to use a database sequence this time.

```
@Entity @Table(name = "post")
public class Post {

    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE)
    private Long id;
}
```

Running the previous test case generates the following output:

```
CALL NEXT VALUE FOR hibernate_sequence
CALL NEXT VALUE FOR hibernate_sequence

DEBUG - Flush is triggered at commit-time

INSERT INTO post (id) VALUES (1, 2)
```

When executing the `persist()` method, Hibernate calls the associated database sequence and fetches an identifier for the newly persisted entity. The actual insert statement is postponed until flush time, which allows Hibernate to take advantage of JDBC batching.

9.2.3.3 Table generator

Because of the mismatch between the identifier generator and the transactional write-behind cache, JPA offers an alternative sequence-like generator that works even when sequences are not natively supported.

A database table is used to hold the latest sequence value, and row-level locking is employed to prevent two concurrent connections from acquiring the same identifier value.

Escaping transactional row-level locking

A database sequence is a non-transactional object because the sequence value allocation happens outside of the transactional context associated with the database connection requesting a new identifier. Database sequences use dedicated locks to prevent concurrent transactions from acquiring the same value, but locks are released as soon as the counter is incremented. This design ensures minimal contention even when the sequence is used concomitantly by multiple concurrent transactions.

Using a database table as a sequence is challenging, as, to prevent two transactions from getting the same sequence value, row-level locking must be used. However, unlike the sequence object locks, the row-level lock is transactional, and, once acquired, it can only be released when the current transaction ends (either committing or rolling back). This would be a terrible scalability issue because a long-running transaction would prevent any other transaction from acquiring a new sequence value.

To cope with this limitation, a separate database transaction is used for fetching a new sequence value. This way, the row-level lock associated with incrementing the sequence counter value can be released as soon as the sequence update transaction ends.

For local transactions, a new transaction means fetching another database connection and committing it after executing the sequence processing logic. This can put additional pressure on the underlying connection pool, especially if there is already a significant contention for database connections.

In a JTA environment, the currently running transaction must be suspended, and the sequence value is fetched in a separate transaction. The JTA transaction manager has to do additional work to accommodate the transaction context switch, and that can also have an impact on the overall application performance.

Without any application-level optimization, the row-level locking approach can become a performance bottleneck if the sequence logic is called way too often.

To continue the previous example, the *post* uses the table generator this time:

```
@Entity @Table(name = "post")
public class Post {

    @Id
    @GeneratedValue(strategy=GenerationType.TABLE)
    private Long id;
}
```

The following output is obtained when inserting a new *post*:

```
SELECT tbl.next_val
FROM hibernate_sequences tbl
WHERE tbl.sequence_name=default
FOR UPDATE

INSERT INTO hibernate_sequences (sequence_name, next_val)
VALUES (default, 1)

UPDATE hibernate_sequences SET next_val=2
WHERE next_val=1 AND sequence_name=default

SELECT tbl.next_val
FROM hibernate_sequences tbl
WHERE tbl.sequence_name=default
FOR UPDATE

UPDATE hibernate_sequences SET next_val=3
WHERE next_val=2 AND sequence_name=default

DEBUG - Flush is triggered at commit-time

INSERT INTO post (id) values (1, 2)
```

The table generator benefits from JDBC batching, but every table sequence update incurs three steps:

- The lock statement is executed to ensure that the same sequence value is not allocated for two concurrent transactions.
- The current value is incremented in the data access layer.
- The new value is saved back to the database and the secondary transaction is committed to release the row-level lock.

Unlike identity columns and sequences, which can increment the sequence in a single request, the table generator entails a significant performance overhead. For this reason, Hibernate comes with a series of optimizers which can improve performance for both sequence and table generators.

Although it is a portable identifier generation strategy, the table generator introduces a serializable execution (the row-level lock), which can hinder scalability. Compared to this application-level sequence generation technique, identity columns and sequences are highly optimized for high concurrency scenarios and should be the preferred choice anyway.

9.2.3.4 Optimizers

As previously mentioned, both the sequence and the table identifier generator have multiple implementations which can improve the performance of the identifier generation process. The sequence and table generators can be split into two categories:

- Legacy implementations (being deprecated since Hibernate 5.0) like `SequenceGenerator`, `SequenceHiLoGenerator` and `MultipleHiLoPerTableGenerator`.
- Newer and more efficient implementations such as `SequenceStyleGenerator` and `TableGenerator`.

These two categories are not compatible, and the application developer must either choose the legacy identifiers or the enhanced ones. Prior to Hibernate 5.0, the legacy identifier generators were provided by default and the application developer could switch to the newer ones by setting the following configuration property:

```
<property name="hibernate.id.new_generator_mappings" value="true"/>
```

Hibernate 5 has decided to drop support for the legacy identifiers and to use the enhanced ones by default.

Among the legacy identifier generators, the `SequenceGenerator` did not offer any optimization, as every new identifier value would require a call to the underlying database sequence. On the other hand, the `SequenceHiLoGenerator` and the `MultipleHiLoPerTableGenerator` offered a *hi/lo* optimization mechanism aimed to reduce the number of calls to a database server. Although these generators are deprecated, the legacy *hi/lo* algorithm is still a valid optimizer even for the newer identifier generators.

9.2.3.4.1 The hi/lo algorithm

The hi/lo algorithm splits the sequences domain into *hi* groups. A *hi* value is assigned synchronously, and every *hi* group is given a maximum number of *lo* entries, that can by assigned off-line without worrying about identifier value conflicts.

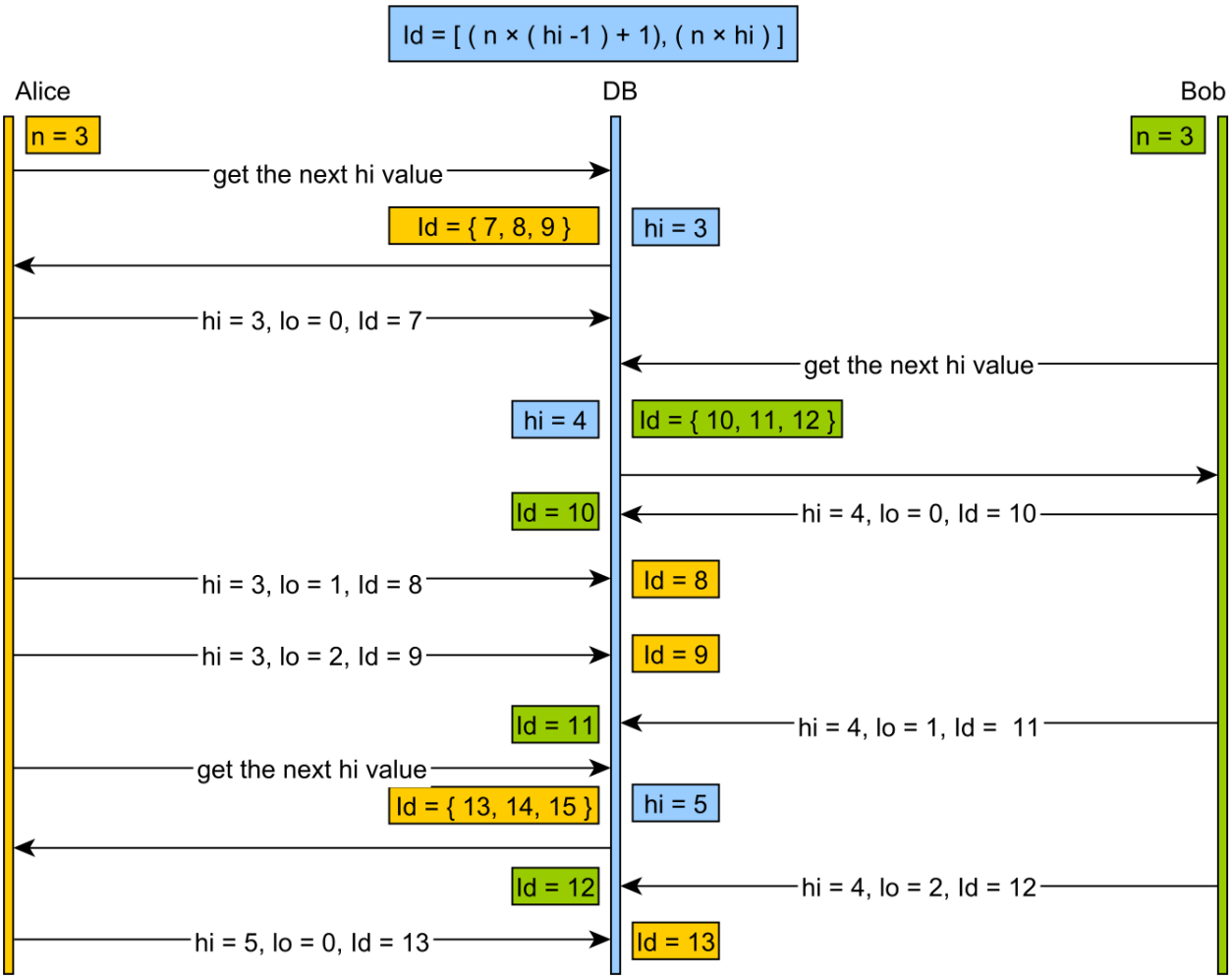

Figure 9.2: The hi/lo algorithm

1. The *hi* token is assigned either by the database sequence or the table generator, so two consecutive calls are guaranteed to see monotonically increasing values.
2. Once a *hi* token is retrieved, the *increment size* (*n - the number of *lo* entries*) defines the range of identifier values a transaction can safely allocate. The identifiers range is bounded by the following interval: $Id = [n \times (hi - 1) + 1, n \times hi]$ and the allocation is done as follows:
   - the current group values start from $n \times (hi - 1) + 1$
   - the *lo* value is taken from the following interval: {$$} [0, n - 1] {$$}
   - by adding the *lo* value to the initial group value, a unique identifier is obtained.
3. When all *lo* values are used, a new *hi* value is fetched, and the cycle continues.

The following example shows how the hi/lo algorithm works in practice. The entity is mapped as follows:

```
@Entity
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "hilo")
    @GenericGenerator(
        name = "hilo",
        strategy = "org.hibernate.id.enhanced.SequenceStyleGenerator",
        parameters = {
            @Parameter(name = "sequence_name", value = "sequence"),
            @Parameter(name = "initial_value", value = "1"),
            @Parameter(name = "increment_size", value = "3"),
            @Parameter(name = "optimizer", value = "hilo")
        }
    )
    private Long id;
}
```

Because the increment size is 3, the following test inserts 4 entities to show the number of database sequence calls.

```
doInJPA(entityManager -> {
    for(int i = 0; i < 4; i++) {
        Post post = new Post();
        entityManager.persist(post);
    }
});
```

Running the previous test generates the following output:

```
CALL NEXT VALUE FOR hilo_seqeunce
CALL NEXT VALUE FOR hilo_seqeunce

INSERT INTO post (id) VALUES (1)
INSERT INTO post (id) VALUES (2)
INSERT INTO post (id) VALUES (3)
INSERT INTO post (id) VALUES (4)
```

The first sequence call is for the first three values, while the second one is generated when reaching the forth entity that needs to be persisted. The more inserts a transaction requires, the better the performance gain from reducing the number of database sequence calls.

Unfortunately, this optimizer has a major limitation. Because the database sequence only assigns group values, all database clients must be aware of this algorithm. If the DBA must insert a row in the table above, he must use the hi/lo algorithm to determine the range of values that she can safely use.

For this reason, Hibernate offers other optimizer algorithms that are interoperable with external clients, unaware of the application-level optimization technique in use.

9.2.3.4.2 The default sequence identifier generator

The JPA identifier generator strategy only specifies the identifier type and not the algorithm used for generating such identifiers.

For the sequence generator, considering the following JPA mapping:

```
@Id
@GeneratedValue(generator = "sequence", strategy=GenerationType.SEQUENCE)
@SequenceGenerator(name = "sequence", allocationSize = 3)
private Long id;
```

Hibernate chooses the `SequenceHiLoGenerator` when the `hibernate.id.new_generator_mappings` configuration property is false. This was the default setting for Hibernate 3 and 4. The legacy `SequenceHiLoGenerator` uses the hi/lo algorithm, and, if the allocation size is greater than one, database interoperability could be compromised (every insert must be done according to the hi/lo algorithm rules).

If the aforementioned configuration property is true (the default setting for Hibernate 5), then the JPA mapping above will use the `SequenceStyleGenerator` instead.

Unlike its previous predecessor, the `SequenceStyleGenerator` uses configurable identifier optimizer strategies, and the application developer can even supply its own optimization implementation.

9.2.3.4.3 The default table identifier generator

Just like with sequences, the JPA table generator mapping can use a legacy or an enhanced generator, depending on the current Hibernate configuration settings:

```
@Id
@GeneratedValue(generator = "table", strategy=GenerationType.TABLE)
@TableGenerator(name = "table", allocationSize = 3)
private Long id;
```

If the `hibernate.id.new_generator_mappings` configuration property is false, then Hibernate will choose the `MultipleHiLoPerTableGenerator`. This generator requires a single table for managing multiple identifiers, and just like `SequenceHiLoGenerator`, it also uses the hi/lo algorithm by default.

When the enhanced identifier generators are activated, Hibernate uses the `TableGenerator` instead, which can also take configurable optimizer strategies.

For both the enhanced sequence and the table identifier generator, Hibernate comes with the following built-in optimizers:

Table 9.8: Hibernate identifier optimizers

Optimizer type	Implementation class	Description
`none`	`NoopOptimizer`	Every identifier is fetched using a new roundtrip to the database
`hi/lo`	`HiLoOptimizer`	It allocates identifiers by using the legacy hi/lo algorithm
`pooled`	`PooledOptimizer`	It is an enhanced version of the hi/lo algorithm which is interoperable with other systems unaware of this identifier generator
`pooled-lo`	`PooledLoOptimizer`	It is a variation of the `pooled` optimizer, the database sequence value representing the *lo* value instead of the *hi* one

By default, the `SequenceStyleGenerator` and `TableGenerator` identifier generators uses the `pooled` optimizer. If the `hibernate.id.optimizer.pooled.prefer_lo` configuration property is set to true, Hibernate will use the `pooled-lo` optimizer by default.

Both the `pooled` and the `pooled-lo` encode the database sequence value into the identifier range boundaries, so allocating a new value using the actual database sequence call does not interfere with the identifier generator allocation process.

9.2.3.4.4 The pooled optimizer

The pooled optimizer can be configured as follows:

```
@Entity
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "pooled")
    @GenericGenerator(
        name = "pooled",
        strategy = "org.hibernate.id.enhanced.SequenceStyleGenerator",
        parameters = {
            @Parameter(name = "sequence_name", value = "sequence"),
            @Parameter(name = "initial_value", value = "1"),
            @Parameter(name = "increment_size", value = "3"),
            @Parameter(name = "optimizer", value = "pooled")
        }
    )
    private Long id;
}
```

The increment size gives the range of values allocated by the sequence generator with one database roundtrip. Although it is inefficient to flush the Persistence Context after every `persist` method call, in this test, the flush outlines when the database sequence was called.

```
doInJPA(entityManager -> {
    for (int i = 0; i < 5; i++) {
        entityManager.persist(new Post());
        entityManager.flush();
    }
    entityManager.unwrap(Session.class).doWork(connection -> {
        try(Statement statement = connection.createStatement()) {
            statement.executeUpdate(
                "INSERT INTO post VALUES NEXT VALUE FOR sequence"
            );
        }
    });
    for (int i = 0; i < 3; i++) {
        entityManager.persist(new Post());
        entityManager.flush();
    }
});
```

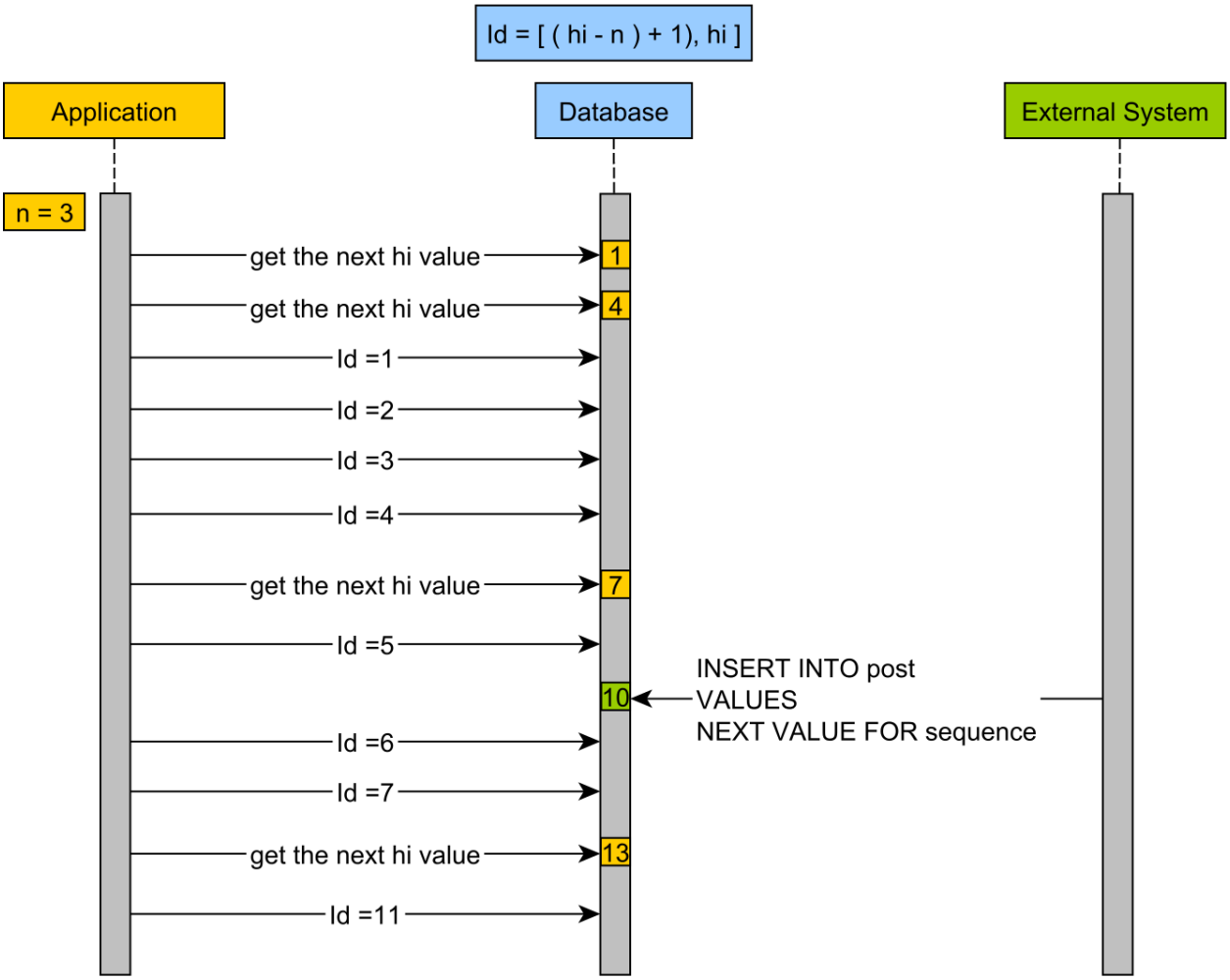

Figure 9.3: The pooled optimizer

If increment size (n) is the number of identifiers within a range, the pooled optimizer will generate identifiers with the following formula: $Id = [(hi - n) + 1, hi]$.

- The first sequence call generates the *lo* value and the second one determines the *hi* value, so the first range of identifiers is {2, 3, 4}.
- When adding the 5th entity, the pooled optimizer calls the sequence again and obtains the next *hi* value, the next identifier range being {5, 6, 7}.
- After inserting the 5th entity, an external system adds a *post* row and assigns the primary key with the value returned by the sequence call.
- The Hibernate application thread resumes and inserts the identifiers 6 and 7.
- The 8th entity requires a new sequence call, and so a new range is allocated {11, 12, 13}.

9.2.3.4.5 The pooled-lo optimizer

By changing the previous mapping to use the `pooled-lo` optimizer, the identifier generation changes as follows:

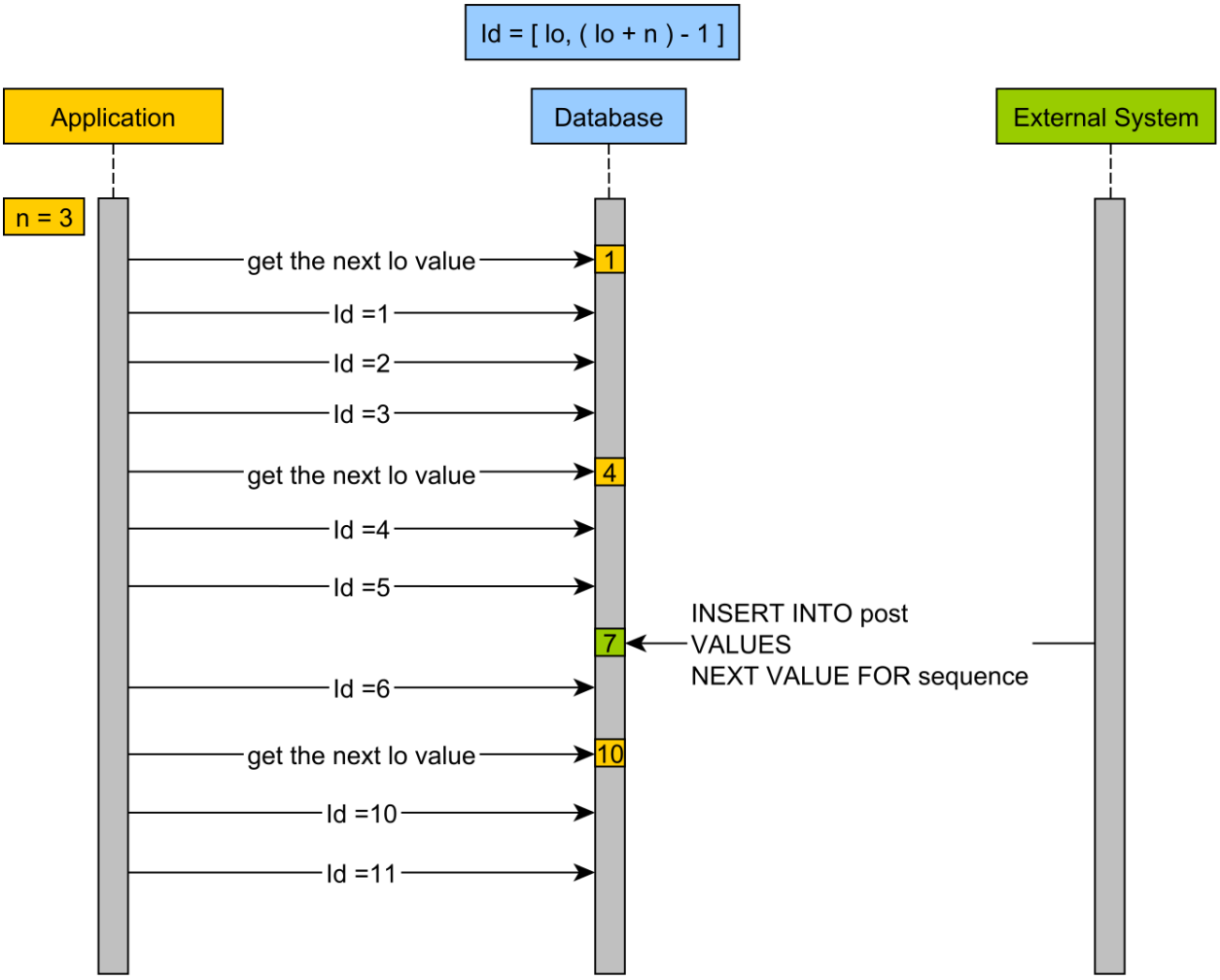

Figure 9.4: The pooled-lo optimizer

If increment size (n) is the number of identifiers within a range, the pooled-lo optimizer will generate identifiers with the following formula: $Id = [lo, (lo + n) - 1]$.

- The first sequence call generates the *lo* value, so the first range of identifiers is {1, 2, 3}.
- When adding the 4th entity, the pooled-lo optimizer calls the sequence and obtains the next *lo* value, the next identifier range being {4, 5, 6}.
- After inserting the 5th entity, an external system adds a *post* row and assigns the primary key with the value returned by the sequence call.
- The Hibernate application thread resumes and inserts the identifier 6.
- The 7th entity requires a new sequence call, and so a new range is allocated {10, 11, 12}.

9.2.3.5 Optimizer gain

To visualize the performance gain of using sequence and table generator optimizers, the following test measures the identifier allocation time when inserting 50 *post* entities and while varying the increment size (1, 5, 10, and 50).

9.2.3.5.1 Sequence generator performance gain

When using a sequence generator with the default pooled optimizer, the following 99th percentile is being recorded:

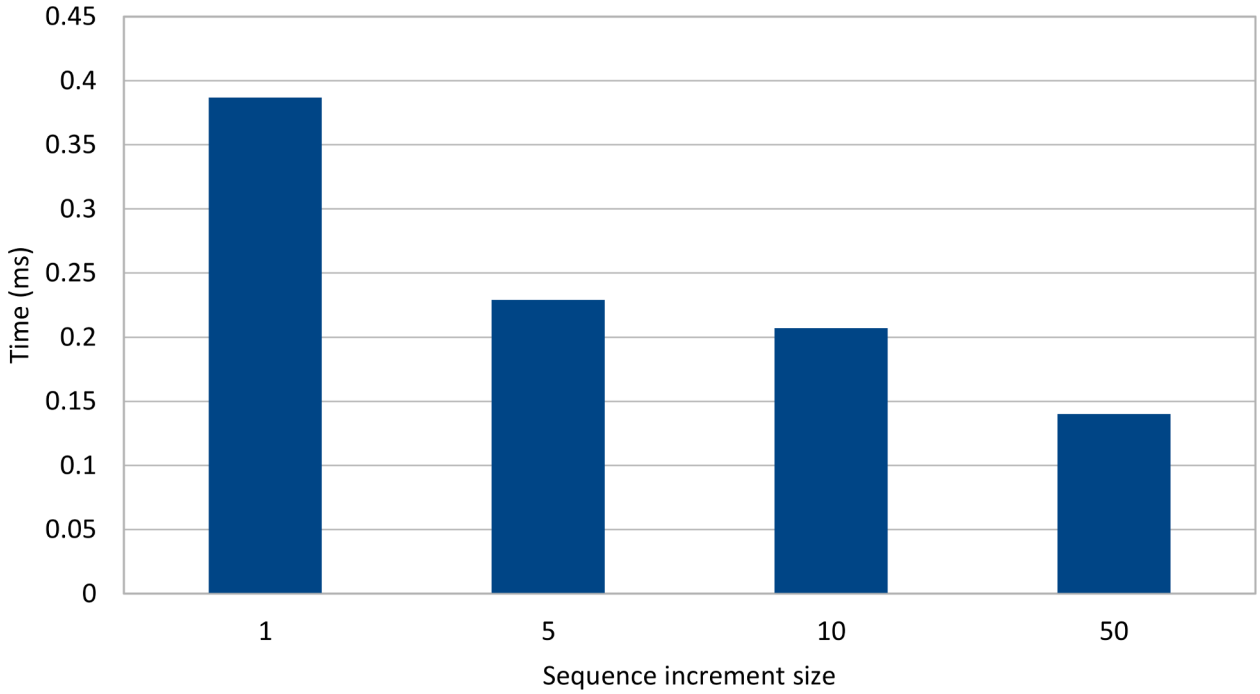

Figure 9.5: Sequence pooled optimizer gain

Database sequences are fast, but, even so, the pooled optimizer manages to reduce the identifier generation time considerably.

For write-intensive applications, the increment size needs to be adjusted according to the number of rows being inserted in one transaction.

9.2.3.5.2 Table generator performance gain

The same test suite is run against a table generator with a pooled optimizer, and the increment size also varies between 1, 5, 10, and 50. Because of the row-level locking and the extra database connection switch overhead, the table generator is less efficient than a database sequence.

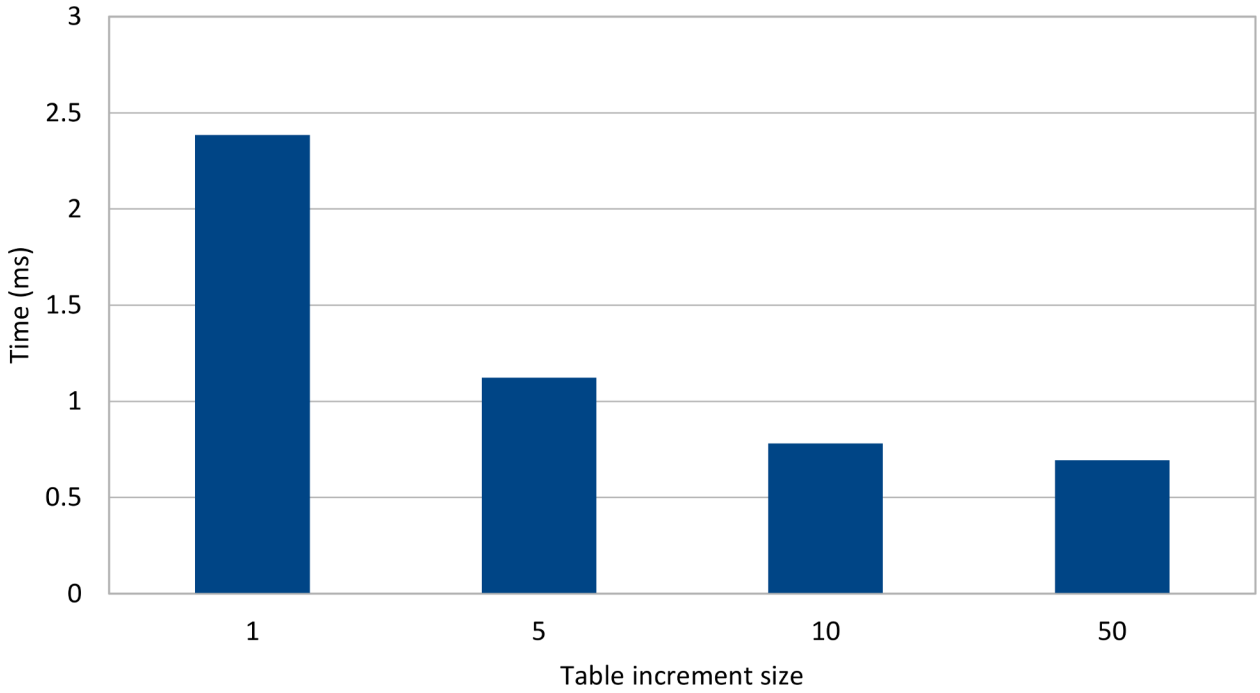

Figure 9.6: Table pooled optimizer gain

Just like with the database sequence, the pooled optimizer managed to reduce the time it took for assigning a new entity identifier.

9.2.3.6 Identifier generator performance

To evaluate the concurrency cost of each identifier generators, the following test measures the time it takes to insert 100 post entities when multiple running threads are involved. JDBC batching is enabled, and the connection pool is adjusted to accommodate the maximum number of database connection required (e.g. 32).

In reality, the application might not be configured with so many database connections, and the table generator connection acquisition cost might be even higher.

The first relational database system under test supports identity columns, so it is worth measuring how the identifier and the table generator compete. Unlike the previous test, this one measures the total time taken for inserting all entities, and not just the identifier allocation time interval.

Each test iteration increases contention by allocating more worker threads that need to execute the same database insert load.

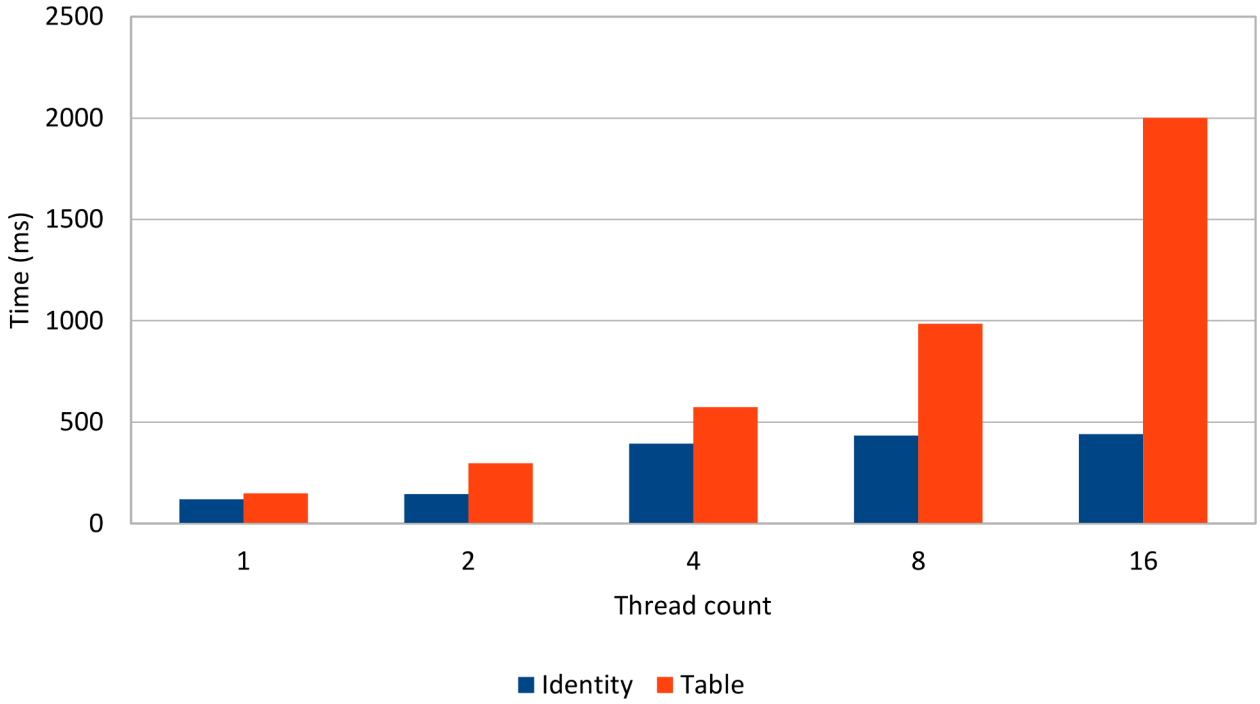

Figure 9.7: Identity vs. Table

Even if it cannot benefit from JDBC batching, the identity generator still manages to outperform the table generator, which uses a pooled optimizer with an increment size of 100.

The more threads are used, the less efficient the table generator becomes. On the other hand, identity columns scale much better with more concurrent transactions.

Even if does not support JDBC batching, native identity columns are still a valid choice, and, in future, Hibernate might even support batch inserts for those as well.

The gap between the sequence and the table generator is even higher because, just like the table generator, the sequence generator can also take advantage of the pooled optimizer as well as JDBC batch inserts.